PROPOSAL

Save AI results to the database

Translate a scenario once, reuse the saved result, and cut token use by about 85%

XenAnalyticTransl - PLC Logic Builder AI layer

Siti Syaziyah | 21 October 2026

Every click re-translates every scenario

Even when nothing has changed since the last run

7

AI calls per click

One per scenario - all seven, every time

~33K

tokens per full run

About 4,700 tokens per scenario

~5

full runs per day

Groq free tier: ~200K tokens a day

Most of these calls repeat work already done. On a free tier that limit is the real constraint on how much we can test.

Translate once, save it, reuse it

The database remembers every result; the AI is called only for what is new

7 AI calls

First launch

Database is empty. Translate all seven and save them.

0 AI calls

Click again, nothing changed

All seven load straight from the database.

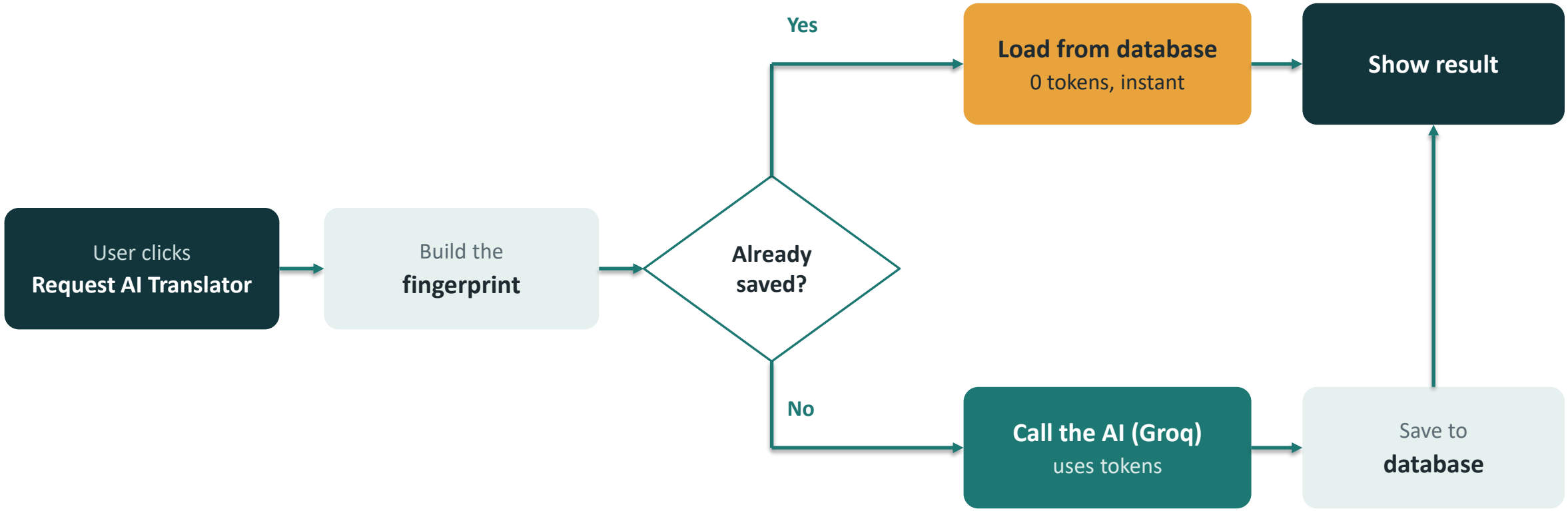
1 AI calls

One scenario edited

Translate that one. Load the other six from the database.

About 85% fewer tokens when one scenario changes: roughly 4,700 tokens instead of 33,000.

How it works, for each scenario



Repeats for all 7 scenarios, then shows a short summary.

A failed AI call is never saved. It shows as an error, so it is retried next time.

The fingerprint decides: reuse or translate

Same inputs give the same code. Change one letter and the code changes

1. Scenario text
2. Scenarios it refers to
3. Property list
4. Model and output type
5. Translation rule version



If anyone edits the translation rules, input 5 changes and old results are translated again instead of reused.

The catch it handles

Operation refers to Auto Run Ready. Edit Auto Run Ready and Operation's answer must change too - even though its own text did not.

Because input 2 is included, both are refreshed.

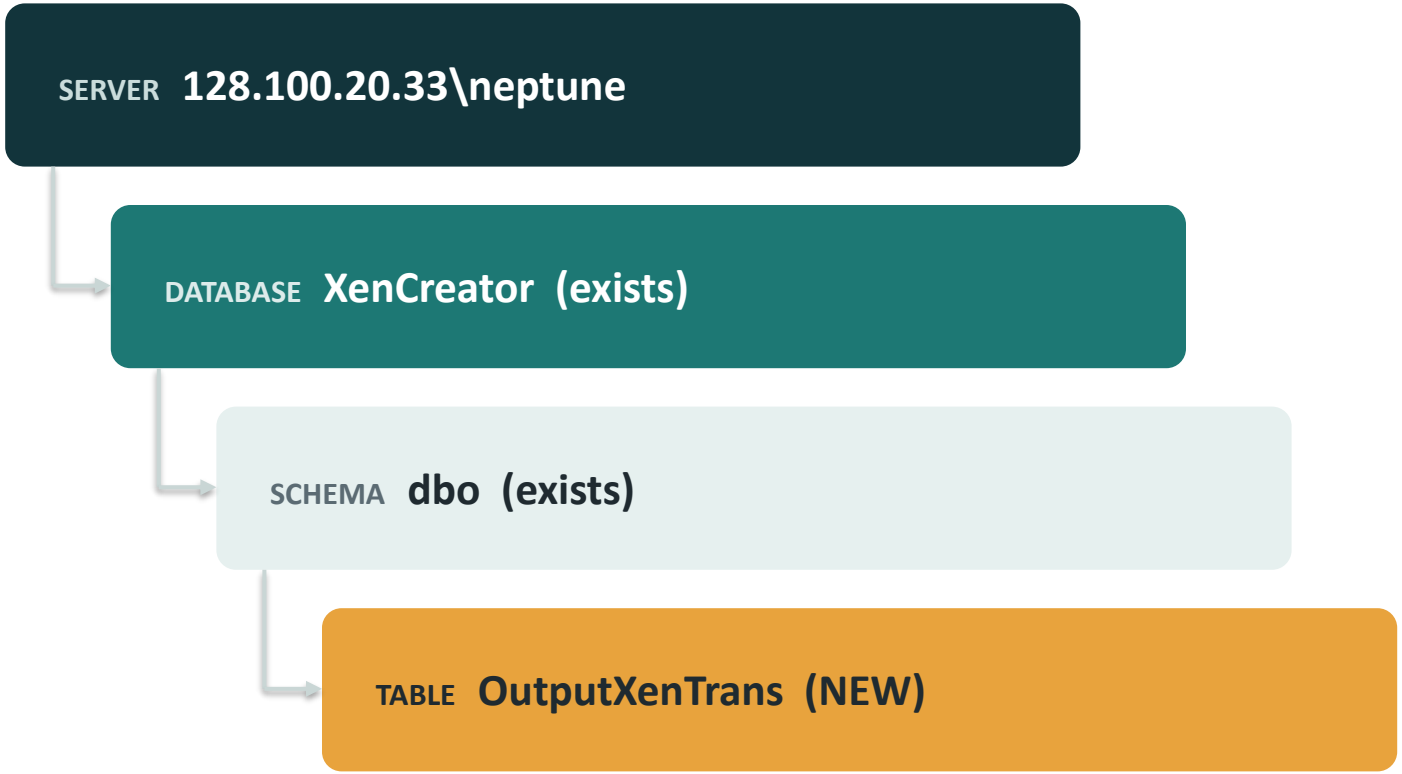
One change needed

Today every request carries all 7 scenarios, so one edit would change all 7 fingerprints. Send only the scenarios actually mentioned.

Bonus: smaller requests on every call.

Where it is stored

A new table inside the existing XenCreator database - not a new database



Full name

`XenCreator.dbo.OutputXenTrans`

"OutputXenTrans.dbo" would read as a new database.

History is kept

Each run adds a row instead of overwriting one.

The app always loads the newest matching row.

Old rows stay, so runs and models can be compared later.

What one saved row holds

Column	Holds	Example
CacheKey	The fingerprint - used to find a match	`7f3a9c21...c91e`
Equipment, ScenarioName	Which scenario this is	`Motor, Trip`
ScenarioText	The scenario exactly as translated	`When Trip Signal is On...`
OutputTarget	Which output was asked for	`Pseudocode`
Provider, Model	Which AI produced it	`Groq, qwen3.8-27b`
Code	The generated pseudocode	`IF Trip Signal == ON THEN...`
Explanation, Warnings	What the AI explained and flagged	`["Trip Alarm never OFF"]`
PromptTokens, CompletionTokens	Tokens used - to measure the saving	`4,422  /  296`
CreatedAt	When it was saved	`2026-10-21 09:14`

The full CREATE TABLE script is in the appendix, ready for the DBA.

What changes in the code

Two new files, four small changes - the AI call and the rules stay as they are

File	Change	Type
AI / ResultStore.cs	Connect to SQL Server; load and save results; build the fingerprint	New
Reference / sql / OutputXenTrans.sql	The CREATE TABLE script for the DBA	New
Studio / MainWindow.xaml.cs	Check the database before calling the AI; send only referenced scenarios	Changed
Studio / MainWindow.xaml	"Ignore saved results" checkbox; saved marker on scenarios	Changed
AI / Models.cs	Flag a result as loaded from the database	Changed
AI / XenAnalyticTransl.AI.csproj	Add Microsoft.Data.SqlClient - the first external package	Changed

Unchanged: LlmTranslator.cs (the AI call) and PromptBuilder.cs (the translation rules).

What the user will see

Status bar after editing one scenario

```
Done: 7 scenario(s) in 24.1s  |  1 translated (4,718 tokens), 6 loaded from database  |  7 clean
```

Scenario list

- DB Operation
- DB System Handle
- DB Run Failed
- **Trip** new
- DB Faults

Rows added to XenCreator.dbo.OutputXenTrans

Id	Scenario	Output	Model	CreatedAt
1	Operation	Pseudocode	qwen3.8-27b	2026-10-21 09:14
2	Trip	Pseudocode	qwen3.8-27b	2026-10-21 09:14
8	Trip	Pseudocode	qwen3.8-27b	2026-10-21 11:02

Row 8 is the re-translated Trip after its edit. Row 2 stays as history.

Four things to agree before building

1 The login stays out of the code

The server uses the rndsa login. Store the connection in an environment variable, like the API keys - or use Windows sign-in if our accounts have access.

2 Permission to create the table

We, or the DBA, need CREATE TABLE rights on XenCreator. The script is ready.

3 Database down? The app still works

If the server cannot be reached, skip the database, call the AI, and show a warning.

4 Saved results are frozen

AI output varies run to run, so a saved answer never improves on its own. An "Ignore saved results" checkbox forces a fresh run.

Next steps

1. **Create the table**
   DBA runs the ready-made script on XenCreator
2. **Agree the login method**
   Environment variable or Windows sign-in
3. **Build and test**
   Against the real Lipico Motor and Valve scenarios
4. **Measure the saving**
   Tokens per run, before and after

THE ASK

Approval to build the database cache.

Expected result: about 85% fewer tokens per test run, so the free tier supports many more tests.

Appendix: CREATE TABLE script

```
CREATE TABLE XenCreator.dbo.OutputXenTrans (
    Id               INT IDENTITY PRIMARY KEY,
    CacheKey         CHAR(64)       NOT NULL,   -- the fingerprint
    Equipment        NVARCHAR(50)   NOT NULL,   -- Motor / Valve
    ScenarioName     NVARCHAR(200)  NOT NULL,
    ScenarioText     NVARCHAR(MAX)  NOT NULL,
    OutputTarget     NVARCHAR(30)   NOT NULL,   -- Pseudocode / Logic Expression / SCL
    Provider         NVARCHAR(100)  NOT NULL,
    Model            NVARCHAR(100)  NOT NULL,
    Code             NVARCHAR(MAX)  NOT NULL,   -- the pseudocode
    Explanation      NVARCHAR(MAX)  NULL,
    Warnings         NVARCHAR(MAX)  NULL,       -- JSON list
    PromptTokens     INT            NULL,
    CompletionTokens INT            NULL,
    ElapsedMs        INT            NULL,
    CreatedAt        DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME()
);
CREATE INDEX IX_OutputXenTrans_CacheKey
    ON XenCreator.dbo.OutputXenTrans (CacheKey);
```